

本小节内容

数组的定义

一维数组在内存中的存储

1 数组的定义

为了存放鞋子，假设你把衣柜最下面的一层分成了 10 个连续的格子。此时，让他人帮你拿鞋子就会很方便，例如你可直接告诉他拿衣柜最下面一层第三个格子中的鞋子。同样假设现在我们有 10 个整数存储在内存中，为方便存取，我们可以借助 C 语言提供的数组，通过一个符号来访问多个元素。

某班学生的学习成绩、一行文字、一个矩阵等数据的特点如下：

- (1) 具有相同的数据类型。
- (2) 使用过程中需要保留原始数据。

C 语言为了方便操作这些数据，提供了一种构造数据类型——数组。所谓数组，是指一组具有相同数据类型的数据的有序集合。

一维数组的定义格式为

```
类型说明符 数组名 [常量表达式];
```

例如，

```
int a[10];
```

定义一个整型数组，数组名为 a，它有 10 个元素。

声明数组时要遵循以下规则：

- (1) 数组名的命名规则和变量名的相同，即遵循标识符命名规则。
- (2) 在定义数组时，需要指定数组中元素的个数，方括号中的常量表达式用来表示元素的个数，即数组长度。
- (3) 常量表达式中可以包含常量和符号常量，但不能包含变量。也就是说，C 语言不允许对数组的大小做动态定义，即数组的大小不依赖于程序运行过程中变量的值。

以下是错误的声明示例（最新的 C 标准支持，但是最好不要这么写）：

```
int n;  
scanf("%d", &n); /* 在程序中临时输入数组的大小 */  
int a[n];
```

数组声明的其他常见错误如下：

- ① float a[0]; /* 数组大小为 0 没有意义 */
- ② int b(2)(3); /* 不能使用圆括号 */
- ③ int k=3, a[k]; /* 不能用变量说明数组大小 */

2 一维数组在内存中的存储

语句 `int mark[100];` 定义的一维数组 mark 在内存中的存放情况如下图所示，每个元素都是

整型元素，占用 4 字节，数组元素的引用方式是“数组名[下标]”，所以访问数组 `mark` 中的元素的方式是 `mark[0],mark[1],...,mark[99]`。注意，没有元素 `mark[100]`，因为数组元素是从 0 开始编号的。

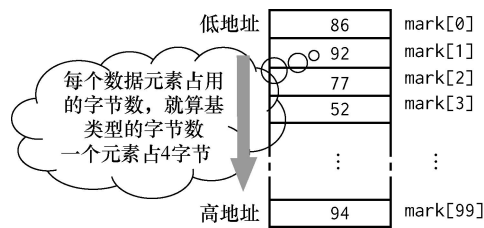


图 一维数组 `mark` 在内存中的存放

下面介绍一维数组的初始化方法。

(1) 在定义数组时对数组元素赋初值。例如，

```
int a[10]={0,1,2,3,4,5,6,7,8,9};
```

不能写成

```
int a[10];a[10]={0,1,2,3,4,5,6,7,8,9}
```

(2) 可以只给一部分元素赋值。例如，

```
int a[10]={0,1,2,3,4};
```

定义 `a` 数组有 10 个元素，但花括号内只提供 5 个初值，这表示只给前 5 个元素赋初值，后 5 个元素的值为 0。

(3) 如果要使一个数组中全部元素的值为 0，那么可以写为

```
int a[10]={0,0,0,0,0,0,0,0,0,0};
```

或

```
int a[10]={0};
```

(4) 在对全部数组元素赋初值时，由于数据的个数已经确定，因此可以不指定数组的长度。例如，

```
int a[]={1,2,3,4,5};
```